DOCUMENTATION

ASSIGNMENT *2*

QUEUES MANAGEMENT APPLICATION USING THREADS AND SYNCHRONIZATION MECHANISMS

STUDENT NAME: Marinescu George
GROUP: 30421

CONTENTS

1. Assignment Objective 3
2. Problem Analysis, Modeling, Scenarios, Use Cases 3
3. Design 4
4. Implementation 4
5. Results 5
6. Conclusions 6
7. Bibliography 6

1. Assignment Objective

i. The main objective of the assignment is to use multithreading to implement a queue management with clients and queues.

ii. The sub-objectives are:
   a. To implement the model, which is comprised of 2 classes: Task (representing clients) and Server (representing queues);
   b. To implement a Scheduler class which is responsible for starting the threads that the queues use;
   c. To implement 2 different strategies for assigning clients to queues;
   d. To implement the SimulationManager class which takes all the components and creates the simulation environment.

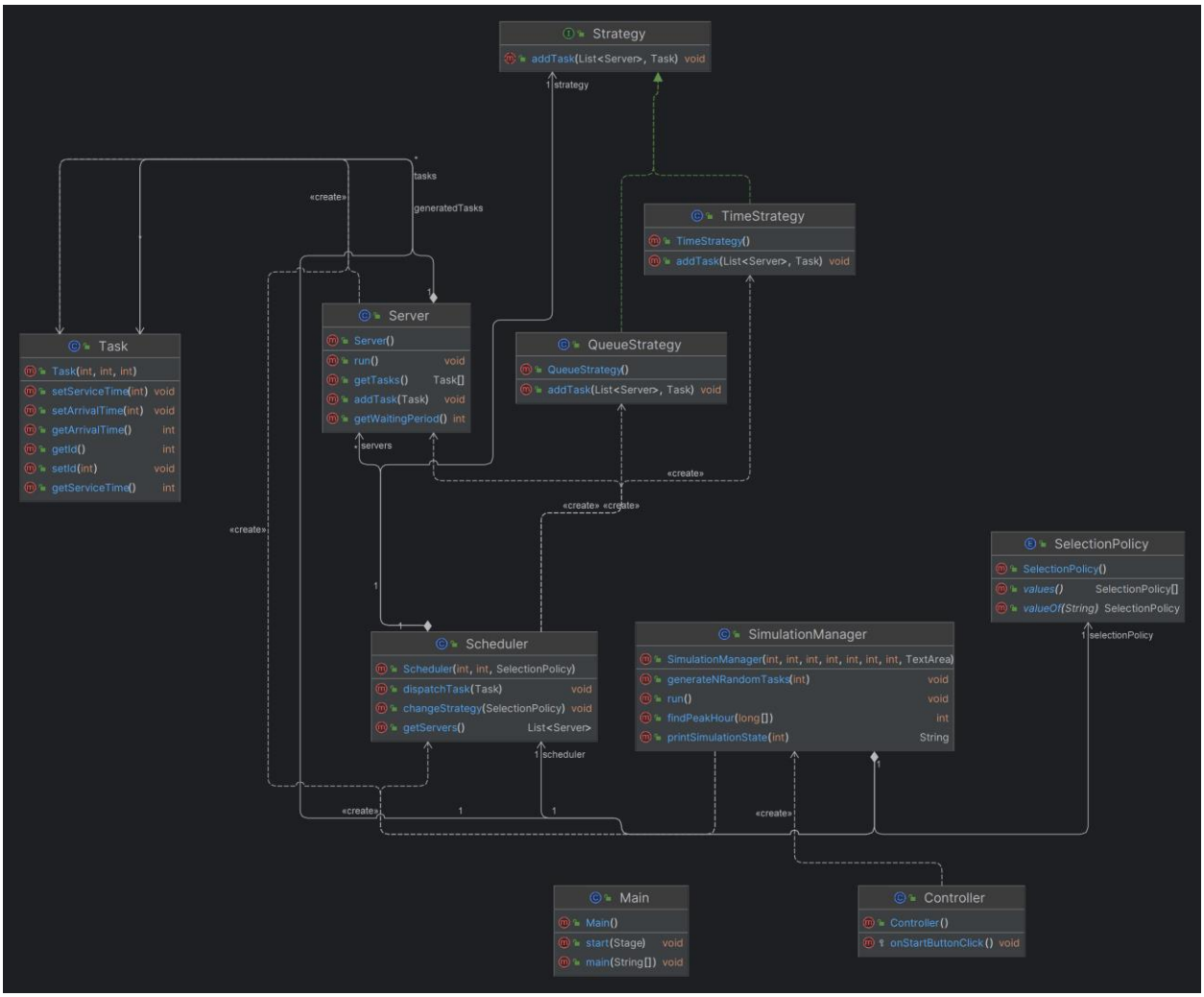
2. Problem Analysis, Modeling, Scenarios, Use Cases

The functional requirements should be presented together with the use cases (use case diagrams and use case description). The use cases' descriptions can be done as a flow-chart or as a list containing the execution steps of each use case.

A. Problem analysis
   We need a way to efficiently manage a scenario where we have multiple clients waiting to be serviced at multiple queues

B. Modeling, scenarios and use cases
   The program displays a user friendly GUI, where the user can input all the necessary data for the simulation : number of clients, number of queues, simulation time, minimum arrival time and maximum arrival time for clients, minimum and maximum service time for clients.
   When the start button is pressed, the simulation begins in the right side of the window.
   At the end of the simulation, average waiting time, average service time and peak hour are displayed.

3. Design



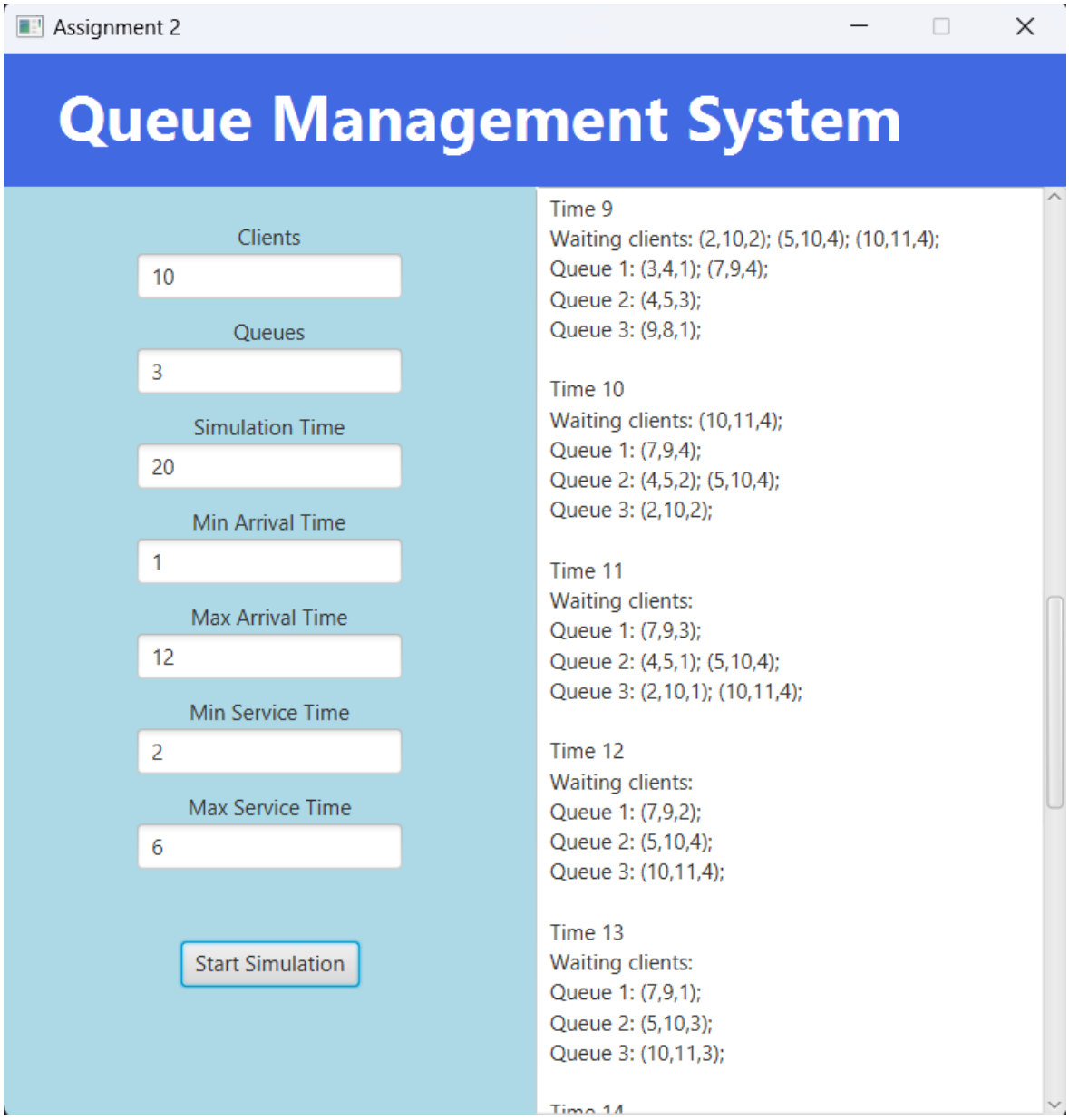
4. Implementation

A. Main package

a. Main – this class initializes the view and launches the program
b. Strategy – this interface declares the AddTask method
c. TimeStrategy – this implements the Strategy interface and implements the AddTask method by adding tasks to the queue that has the lowest waiting time
d. QueueStrtegy – this also implements the Strategy interface and does AddTask with respect to the queue that hass the lowest number of clients waiting
e. Scheduler – this class starts the threads of each Server and selects the appropriate selection policy for tasks
f. SelectionPolicy – this is the enum that contains the selection policies for tasks
g. SimulationManager – this class initializes the simulation, creates random tasks, and starts the main thread of the simulation, together with printing the output

B. Model package
    a. Task – this class represents a client
    b. Server – this class represents a queue. It also contains the run method, which deals with servicing the clients efficiently
C. View
    a. hello-view – the GUI implemented in JavaFX
D. Controller package
    a. Controller – takes data from the view and starts the SimulationManager with it

5. Results



6. Conclusions

This assignment allowed me to work with the concept of multithreading first-hand, and now I am much more familiar with it. Also, it is the first time I am doing a simulation project.
For future development, error handling should be implemented for user safety.

7. Bibliography

1. Stack Overflow